ML Exam — Question & Model Answer Reference

TOPIC 1 — Simple Linear Regression (Week 1)

Dataset: `data/Advertising.csv`

Using the Advertising dataset, fit a simple linear regression to predict `sales` from `TV` spend. Print the intercept, coefficient, R^2, and predict sales when TV = 150.

Model Answer:

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf

df = pd.read_csv('data/Advertising.csv')

result = smf.ols('sales ~ TV', data=df).fit()
print(result.summary())
print("Intercept:", result.params['Intercept'])
print("TV coef:", result.params['TV'])
print("R²:", result.rsquared)
print("Prediction at TV=150:", result.predict(pd.DataFrame({'TV': [150]})))
```

Interpretation: The coefficient on TV means that for every $1000 increase in TV spend, sales increase by ~0.047 units on average. R^2 tells us how much variance in sales is explained by TV alone.

TOPIC 2 — Multiple Linear Regression (Week 1)

Dataset: `data/Advertising.csv`

Fit a multiple linear regression predicting `sales` from `TV`, `radio`, and `newspaper`. Print adjusted R^2, the F-statistic, and identify which predictors are statistically significant ($p < 0.05$).

Model Answer:

```
import pandas as pd
import statsmodels.formula.api as smf

df = pd.read_csv('data/Advertising.csv')

result = smf.ols('sales ~ TV + radio + newspaper', data=df).fit()
print(result.summary())
print("Adj R²:", result.rsquared_adj)
print("F-stat:", result.fvalue)
print("P-values:\n", result.pvalues)
```

```
# Significant predictors
sig = result.pvalues[result.pvalues < 0.05]
print("Significant predictors:", sig.index.tolist())
```

Interpretation: TV and radio are significant ($p < 0.05$); newspaper is not. The F-statistic tests whether at least one predictor is useful. Adjusted R^2 penalises for adding useless predictors.

TOPIC 3 — Polynomial Regression (Week 2)

Dataset: `data/auto.csv`

Fit polynomial regression models of degree 1 through 5 to predict `mpg` from `horsepower`. Use 10-fold cross-validation to find the best degree by MSE. Plot the CV-MSE against degree.

Model Answer:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score, KFold
from sklearn.pipeline import Pipeline

df = pd.read_csv('data/auto.csv', na_values='?').dropna()
X = df[['horsepower']]
y = df['mpg']

kf = KFold(n_splits=10, shuffle=True, random_state=1)
mse_scores = []

for degree in range(1, 6):
    pipe = Pipeline([
        ('poly', PolynomialFeatures(degree=degree, include_bias=False)),
        ('lr', LinearRegression())
    ])
    scores = cross_val_score(pipe, X, y, cv=kf, scoring='neg_mean_squared_error')
    mse_scores.append(-scores.mean())
    print(f"Degree {degree}: MSE = {-scores.mean():.2f}")

plt.plot(range(1, 6), mse_scores, '-o')
plt.xlabel('Degree'); plt.ylabel('CV MSE')
plt.title('Polynomial Degree Selection'); plt.show()

best_degree = np.argmin(mse_scores) + 1
print("Best degree:", best_degree)
```

Interpretation: We choose the degree that minimises CV-MSE. A degree-2 polynomial typically suffices for mpg vs horsepower — beyond that, gains are marginal and variance increases.

TOPIC 4 — Logistic Regression (Week 3)

Dataset: `data/default.csv`

Predict credit `default` (Yes/No) from `balance` and `income`. Encode the target, fit a logistic regression, and print the confusion matrix and classification report. Also print the predicted probability of defaulting for someone with balance=2000 and income=40000.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)

X = df[['balance', 'income']].values
y = df['default_yes'].values

lr = LogisticRegression(max_iter=1000)
lr.fit(X, y)

y_pred = lr.predict(X)
print(confusion_matrix(y, y_pred))
print(classification_report(y, y_pred))

# Predict probability for new observation
prob = lr.predict_proba([[2000, 40000]])
print("P(default | balance=2000, income=40000):", prob[0][1])
```

Interpretation: The confusion matrix shows `[[TN, FP], [FN, TP]]`. High specificity (TN rate) but potentially low sensitivity (TP rate) is common when the positive class is rare (few defaults).

TOPIC 5 — LDA (Week 3)

Dataset: `data/smarket.csv`

Using Lag1 and Lag2 as predictors, train an LDA model on years < 2005 and test on years ≥ 2005 to predict market `Direction` (Up/Down). Print the confusion matrix and overall accuracy.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv('data/smarket.csv')
df['Up'] = (df['Direction'] == 'Up').astype(int)

X = df[['Lag1', 'Lag2']].values
y = df['Up'].values
train = df['Year'] < 2005

lda = LinearDiscriminantAnalysis()
lda.fit(X[train], y[train])

y_pred = lda.predict(X[~train])
print("Priors:", lda.priors_)
print("Confusion matrix:\n", confusion_matrix(y[~train], y_pred))
print("Accuracy:", accuracy_score(y[~train], y_pred))

# Custom threshold
probs = lda.predict_proba(X[~train])[:, 1]
y_pred_custom = (probs >= 0.55).astype(int)
print("Custom threshold accuracy:", accuracy_score(y[~train], y_pred_custom))
```

Interpretation: LDA assumes features are normally distributed with equal covariance across classes. The prior reflects the proportion of Up vs Down days in training data. Raising the threshold makes the model more conservative about predicting "Up".

TOPIC 6 — QDA (Week 3)

Dataset: `data/smarket.csv`

Fit a QDA model using the same Lag1/Lag2 train/test split as Topic 5. Compare QDA accuracy to LDA and explain when QDA is preferred.

Model Answer:

```
import pandas as pd
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv('data/smarket.csv')
df['Up'] = (df['Direction'] == 'Up').astype(int)

X = df[['Lag1', 'Lag2']].values
y = df['Up'].values
train = df['Year'] < 2005
```

```
qda = QuadraticDiscriminantAnalysis()
qda.fit(X[train], y[train])

y_pred = qda.predict(X[~train])
print("Confusion matrix:\n", confusion_matrix(y[~train], y_pred))
print("QDA Accuracy:", accuracy_score(y[~train], y_pred))
```

Interpretation: QDA allows each class to have its own covariance matrix, making it more flexible than LDA. QDA is preferred when the decision boundary is non-linear or when class covariances differ substantially. However, it requires more data to estimate the additional parameters.

TOPIC 7 — KNN Classifier (Week 3)

Dataset: `data/default.csv`

Fit a KNN classifier (k=5) to predict credit default from `balance` and `income`. Print the confusion matrix. Then find the best K (1–20) by comparing training accuracy.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)

X = df[['balance', 'income']].values
y = df['default_yes'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KNN with k=5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_scaled, y)
y_pred = knn.predict(X_scaled)
print("Confusion matrix:\n", confusion_matrix(y, y_pred))

# Find best K
accuracies = []
for k in range(1, 21):
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_scaled, y)
    accuracies.append(accuracy_score(y, knn_k.predict(X_scaled)))
```

```
best_k = accuracies.index(max(accuracies)) + 1
print("Best K:", best_k)
```

Interpretation: KNN is a non-parametric classifier — it makes no assumptions about the data distribution. Scaling features is essential so that `balance` (large values) does not dominate `income`. Lower K = more flexible (risk of overfitting); higher K = smoother boundary.

TOPIC 8 — K-Fold Cross-Validation & LOOCV (Week 4)

Dataset: `data/auto.csv`

Using the auto dataset, compare the CV-MSE for a linear and a quadratic regression of `mpg` on `horsepower` using (a) LOOCV and (b) 10-fold CV.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import cross_val_score, KFold

df = pd.read_csv('data/auto.csv', na_values='?').dropna()
X = df[['horsepower']].values
y = df['mpg'].values

lr = LinearRegression()

for label, cv in [('LOOCV', KFold(n_splits=len(X))),
                  ('10-Fold', KFold(n_splits=10, shuffle=True, random_state=1))]:
    print(f"\n--- {label} ---")
    for degree in [1, 2]:
        poly = PolynomialFeatures(degree=degree)
        X_p = poly.fit_transform(X)
        scores = cross_val_score(lr, X_p, y, cv=cv,
scoring='neg_mean_squared_error')
        print(f"  Degree {degree}: MSE = {-scores.mean():.2f}")
```

Interpretation: LOOCV gives a nearly unbiased estimate of test error but is computationally expensive. 10-fold CV is faster and has lower variance. Both should agree that the quadratic model reduces MSE significantly over the linear one.

TOPIC 9 — Bootstrap (Week 4)

Dataset: `data/default.csv`

Use bootstrap (100 iterations) to estimate the standard error of the intercept and `balance` coefficient in a logistic regression of `default_yes ~ balance + income`.

Model Answer:

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf

df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int)

params_list = []
for i in range(100):
    sample = df.sample(len(df), replace=True)
    result = smf.logit('default_yes ~ balance + income', data=sample).fit(disp=0)
    params_list.append(result.params)

boot_df = pd.DataFrame(params_list)
print("Bootstrap Standard Errors:")
print(boot_df.std())

# Compare to model standard errors
result_full = smf.logit('default_yes ~ balance + income', data=df).fit()
print("\nModel Standard Errors:")
print(result_full.bse)
```

Interpretation: Bootstrap SEs do not rely on model assumptions (like normality of errors). If they agree with the model's `bse`, our model assumptions hold. If they differ significantly, the model SEs may be unreliable.

TOPIC 10 — PCA & PCR (Week 5)

Dataset: `data/hitters.csv`

Perform PCA on the numeric features predicting `Salary`. Print cumulative explained variance. Then use PCR (Principal Component Regression) with 10-fold CV to find how many components minimise MSE.

Model Answer:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import scale
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, KFold

df = pd.read_csv('data/hitters.csv').dropna().drop('Unnamed: 0', axis=1)
y = df['Salary'].values
X = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
axis=1).astype('float64')
```

```
# PCA
pca = PCA()
X_reduced = pca.fit_transform(scale(X))

cum_var = np.cumsum(np.round(pca.explained_variance_ratio_, 4) * 100)
print("Cumulative variance explained:")
for i, v in enumerate(cum_var, 1):
    print(f"  PC{i}: {v:.1f}%")

# PCR via CV
kf = KFold(n_splits=10, shuffle=True, random_state=1)
lr = LinearRegression()
mse = []
for i in range(1, X_reduced.shape[1] + 1):
    scores = cross_val_score(lr, X_reduced[:, :i], y,
                             scoring='neg_mean_squared_error', cv=kf)
    mse.append(-scores.mean())

best_m = np.argmin(mse) + 1
print(f"\nBest number of PCs: {best_m}, CV-MSE: {mse[best_m-1]:.2f}")

plt.plot(range(1, len(mse)+1), mse, '-o')
plt.xlabel('Number of PCs'); plt.ylabel('CV MSE'); plt.show()
```

Interpretation: PCR reduces dimensionality by projecting onto PCs that explain the most variance. The best M PCs chosen by CV balance bias and variance. Note: PCs are linear combinations of original features, so interpretability is reduced.

TOPIC 11 — Ridge Regression (Week 5)

Dataset: `data/hitters.csv`

Fit a Ridge regression to predict `Salary`. Use `StandardScaler` to scale features, then `RidgeCV` (10-fold, 1000 alphas) to find the best regularisation. Evaluate on a 30% test split and print test MSE and top 3 coefficients by magnitude.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import Ridge, RidgeCV
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/hitters.csv').dropna().drop('Unnamed: 0', axis=1)
y = df['Salary'].values
X = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
axis=1).astype('float64')
```

```
# Scale using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
random_state=42)

# Find best alpha with RidgeCV
rcv = RidgeCV(alphas=np.linspace(0.01, 100, 1000), cv=10)
rcv.fit(X_train, y_train)
print("Best alpha:", rcv.alpha_)

# Refit Ridge with best alpha and evaluate
ridge = Ridge(alpha=rcv.alpha_)
ridge.fit(X_train, y_train)

mse = mean_squared_error(y_test, ridge.predict(X_test))
print(f"Test MSE: {mse:.2f}")

# Top 3 coefficients
coef_series = pd.Series(ridge.coef_,
index=X.columns).abs().sort_values(ascending=False)
print("Top 3 predictors:\n", coef_series.head(3))
```

Interpretation: Ridge shrinks coefficients toward zero (L2 penalty: $\lambda\Sigma\beta^2$) but never exactly to zero — all features are kept. `StandardScaler` ensures all features are on the same scale before penalisation. A larger alpha = stronger shrinkage = simpler model.

TOPIC 12 — Lasso Regression (Week 5)

Dataset: `data/hitters.csv`

Fit a Lasso model to predict `Salary`. Use `StandardScaler`, then search for the best alpha over a grid. Print how many coefficients are exactly zero (i.e., features excluded by Lasso) and the test MSE.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import Lasso, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/hitters.csv').dropna().drop('Unnamed: 0', axis=1)
y = df['Salary'].values
X = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
axis=1).astype('float64')
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
random_state=42)

# LassoCV to find best alpha
lasso_cv = LassoCV(alphas=np.linspace(0.01, 100, 1000), cv=10, max_iter=100000)
lasso_cv.fit(X_train, y_train)
print("Best alpha:", lasso_cv.alpha_)

# Refit and evaluate
lasso = Lasso(alpha=lasso_cv.alpha_, max_iter=100000)
lasso.fit(X_train, y_train)

mse = mean_squared_error(y_test, lasso.predict(X_test))
print(f"Test MSE: {mse:.2f}")

coef_series = pd.Series(lasso.coef_, index=X.columns)
print(f"\nCoefficients set to zero: {(coef_series == 0).sum()}")
print("Non-zero features:\n", coef_series[coef_series != 0])
```

Interpretation: Lasso uses an L1 penalty (λΣ|β|) which can shrink coefficients exactly to zero, performing automatic feature selection. This makes Lasso preferable over Ridge when you suspect only a few predictors truly matter.

TOPIC 13 — Step Functions / Piecewise Constant (Week 6–7)

Dataset: `data/wage.csv`

Fit a piecewise constant (step function) regression of `wage` on `age` using 4 bins. Print the model summary and produce a scatter plot with the step function overlaid.

Model Answer:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

df = pd.read_csv('data/wage.csv')

# Cut age into 4 bins
df_cut, bins = pd.cut(df.age, 4, retbins=True, right=True)

# Create dummies, add constant, drop first category (reference)
dummies = sm.add_constant(pd.get_dummies(df_cut))
dummies = dummies.drop(dummies.columns[1], axis=1)

fit = sm.GLM(df.wage, dummies.astype(int)).fit()
```

```
print(fit.summary())

# Predictions on a grid
age_grid = np.arange(df.age.min(), df.age.max()).reshape(-1, 1)
bin_mapping = np.digitize(age_grid.ravel(), bins)
X_test = sm.add_constant(pd.get_dummies(bin_mapping).drop(1, axis=1)).astype(int)
pred = fit.predict(X_test)

# Plot
plt.scatter(df.age, df.wage, alpha=0.2, edgecolor='k', facecolor='None')
plt.plot(age_grid, pred, color='blue', lw=2)
plt.xlabel('Age'); plt.ylabel('Wage')
plt.title('Step Function Regression'); plt.show()

# Shortcut formula version
import statsmodels.formula.api as smf
smf.ols('wage ~ pd.cut(age, 4)', data=df).fit().summary()
```

Interpretation: Step functions fit a constant within each bin. The intercept is the average wage for the youngest age group; other coefficients represent the average additional wage for each subsequent group.

TOPIC 14 — Regression Splines (Week 6–7)

Dataset: `data/wage.csv`

Fit three spline models to `wage ~ age`: (1) cubic spline with knots at 25, 40, 60; (2) cubic spline with df=6; (3) natural spline with df=4. Plot all three predictions on one graph.

Model Answer:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from patsy import dmatrix

df = pd.read_csv('data/wage.csv')
age_grid = np.arange(df.age.min(), df.age.max()).reshape(-1, 1)

# 1. Cubic spline with specified knots
t1 = dmatrix("bs(df.age, knots=(25,40,60), degree=3, include_intercept=False)",
             {"df.age": df.age}, return_type='dataframe')
fit1 = sm.GLM(df.wage, t1).fit()
pred1 = fit1.predict(dmatrix("bs(age_grid, knots=(25,40,60),
include_intercept=False)",
                             {"age_grid": age_grid}, return_type='dataframe'))

# 2. Cubic spline with df=6 (auto knots at 25th/50th/75th percentile)
t2 = dmatrix("bs(df.age, df=6, include_intercept=False)",
             {"df.age": df.age}, return_type='dataframe')
```

```
fit2 = sm.GLM(df.wage, t2).fit()
pred2 = fit2.predict(dmatrix("bs(age_grid, df=6, include_intercept=False)",
                             {"age_grid": age_grid}, return_type='dataframe'))

# 3. Natural spline with df=4
t3 = dmatrix("cr(df.age, df=4)", {"df.age": df.age}, return_type='dataframe')
fit3 = sm.GLM(df.wage, t3).fit()
pred3 = fit3.predict(dmatrix("cr(age_grid, df=4)",
                             {"age_grid": age_grid}, return_type='dataframe'))

plt.scatter(df.age, df.wage, alpha=0.1, facecolor='None', edgecolor='k')
plt.plot(age_grid, pred1, 'b', label='Knots at 25,40,60')
plt.plot(age_grid, pred2, 'r', label='df=6')
plt.plot(age_grid, pred3, 'g', label='Natural spline df=4')
[plt.axvline(x=k, linestyle='--', color='b', lw=1) for k in [25, 40, 60]]
plt.legend(); plt.xlabel('Age'); plt.ylabel('Wage'); plt.show()
```

Interpretation: `bs()` fits a B-spline (unconstrained at boundaries); `cr()` fits a natural cubic spline (linear beyond boundary knots, reducing variance). More knots = more flexibility = lower bias but higher variance.

TOPIC 15 — GAM (Week 6–7)

Dataset: Breast cancer (sklearn built-in)

Fit a Logistic GAM to classify tumours as malignant/benign using the first 6 features. Print accuracy, display the summary, and plot partial dependence for each feature.

Model Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
from pygam import LogisticGAM
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)[
    ['mean radius', 'mean texture', 'mean perimeter',
     'mean area', 'mean smoothness', 'mean compactness']]
y = data.target

gam = LogisticGAM().fit(X, y)
gam.summary()
print("Accuracy:", gam.accuracy(X, y))

# Partial dependence plots
fig, axs = plt.subplots(1, 6, figsize=(28, 4))
for i, ax in enumerate(axs):
    XX = gam.generate_X_grid(term=i)
    pdep, confi = gam.partial_dependence(term=i, width=0.95)
    ax.plot(XX[:, i], pdep)
```

```
    ax.plot(XX[:, i], confi[:, 0], 'k--')
    ax.plot(XX[:, i], confi[:, 1], 'k--')
    ax.set_title(X.columns[i])
plt.tight_layout(); plt.show()
```

Interpretation: A GAM extends GLMs by replacing each linear term $\beta_j x_j$ with a smooth function $f_j(x_j)$. Each partial dependence plot shows the marginal effect of one feature holding others constant. GAMs are more interpretable than black-box models while still capturing non-linearity.

TOPIC 16 — KNN Regression (Week 6–7)

Dataset: `data/wage.csv`

Fit a KNN regressor to predict `wage` from `age`. Use `GridSearchCV` with 5-fold CV to find the best K from 1–30. Print the best K and test RMSE.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import mean_squared_error
from math import sqrt

df = pd.read_csv('data/wage.csv')
X = df[['age']]
y = df['wage']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)

# Grid search for best K
model = GridSearchCV(
    KNeighborsRegressor(),
    param_grid={"n_neighbors": list(range(1, 31))},
    cv=5,
    scoring='neg_mean_squared_error'
)
model.fit(X_train, y_train)

best_k = model.best_params_['n_neighbors']
print("Best K:", best_k)

y_pred = model.predict(X_test)
rmse = sqrt(mean_squared_error(y_test, y_pred))
print(f"Test RMSE: {rmse:.2f}")
```

Interpretation: KNN regression predicts the average wage of the K nearest age values. Small K = very wiggly fit (overfitting); large K = overly smooth (underfitting). CV selects the K that generalises best.

End of Reference — Topics 1–16 covering Weeks 1–7